

FASE 0 · O AGENTE EM CÓDIGO PURO

Aula 0.3

O loop de agente

O agente não é o modelo. O agente é o loop. A volta manual da aula anterior vira um laço que encadeia ferramentas sozinho, com condições de parada explícitas.

Série: Engenharia de IA Aplicada · Agentes e RAG em Produção

Projeto: Núcleo de Agentes Operacionais

Duração de estudo: cerca de 1h30 · 25 min de teoria e 55 de código

Pré-requisito: Aulas 0.1 e 0.2 (tag **aula-0.2**)

1. Objetivo de aprendizagem

Ao final desta aula você será capaz de escrever, do zero e sem framework, o laço que faz um LLM encadear várias ferramentas sozinho até chegar à resposta final, com condições de parada explícitas e limite de iteração, funcionando igual na Anthropic e na OpenAI.

2. Por que esta aula importa

Até a aula 0.2 você tinha uma chamada de LLM com mãos, mas de uma volta só. Você decidia parar. Isso ainda não é um agente.

A FRASE DA AULA

O agente não é o modelo. O agente é o loop. Trocar de modelo muda a qualidade das decisões. Não muda o que faz o sistema ser um agente: o laço em que o modelo decide, você executa, devolve a observação, e ele decide de novo com informação nova.

Quando um agente em produção entra em ciclo, gasta uma fortuna numa única sessão, chama a mesma ferramenta seis vezes ou trava sem responder, o problema quase nunca está no prompt. Está no laço, na condição de parada ou no que você devolveu como observação. Quem só montou agente em ferramenta visual ou em framework não sabe olhar aí, porque nunca viu esse laço escrito. Depois de hoje, você escreveu.

3. Teoria

3.1 A anatomia universal do agent loop

Momento	O que é
Contexto	Tudo que o modelo enxerga agora: na prática, a lista de mensagens acumulada.
Decisão	O modelo escolhe entre pedir uma ou mais ferramentas, ou responder em texto.
Ação	O seu código executa o que foi pedido.
Observação	O resultado volta para o histórico e vira contexto da próxima volta.

Repare no detalhe que muda tudo: **a decisão de sair do laço é do modelo, não sua.** Você não escreve `if pergunta_sobre_lead: buscar_lead()`. Você oferece as ferramentas e responde ao que ele pedir. Essa inversão de controle é a fronteira entre um workflow e um agente.

3.2 O histórico é o estado

Não existe memória mágica. O modelo não lembra da volta anterior: ele relê a conversa inteira a cada chamada, com um item a mais. A lista de mensagens **é** o estado do agente. Hoje ela vive na memória e morre com o processo; persistir essa lista é exatamente o que o checkpointing do LangGraph faz na Fase 1. E como cada volta reenvia tudo, cada volta custa mais que a anterior: o número de iterações é métrica direta de custo.

3.3 As quatro condições de parada

Um laço sem condição de parada explícita é um incidente esperando acontecer, e em agentes é um incidente com fatura.

- **Resposta natural.** O modelo respondeu sem pedir ferramenta. É o caso feliz e o único que representa sucesso.
- **Limite de iterações.** O teto duro. Não é paranoia, é orçamento. Implementamos com `for` e não com `while True`: o teto fica embutido na estrutura, e é impossível esquecê-lo.
- **Erro irre recuperável.** Provider fora do ar, chave expirada. O agente não tem como contornar; a exceção sobe. A aula 0.6 separa esse caso do erro de ferramenta, que o agente contorna.
- **Interrupção humana.** Ainda não implementada. Vira primitivo de verdade no LangGraph, na Fase 1.

ATENÇÃO

Bater o limite de iterações **não é sucesso**. Devolver a última frase do modelo como se fosse a resposta final é a forma mais comum de mascarar um agente quebrado. Por isso o nosso laço devolve o motivo da parada explicitamente.

3.4 Pedidos paralelos

O modelo pode pedir **várias** ferramentas numa única volta quando elas são independentes, por exemplo buscar dois leads para comparar. Os dois providers fazem isso. Consequência de correção: se o seu laço assume um pedido por volta, ele descarta os outros em silêncio, e o agente segue com informação faltando sem que nada quebre. A forma de devolver também difere: a Anthropic espera **todos** os resultados num único turno de user; a OpenAI espera **uma** mensagem de role tool por pedido.

3.5 Onde a diferença entre providers morre

Se o laço tiver que saber a diferença entre os dialetos, você acaba com dois laços. Então ela morre na camada de provider, exatamente como a `chat()` da aula 0.1 já faz para texto. Duas funções novas em `providers.py` garantem que o laço receba sempre o mesmo objeto neutro e nunca veja dialeto.

4. Mão na massa

Esta aula acrescenta código de produto ao Núcleo: uma extensão em `nucleo/providers.py` (a `chat()` da 0.1 fica intocada), o arquivo novo `nucleo/agente.py` e os testes em `tests/test_agente.py`.

Passo 1 — O formato neutro e a chamada com ferramentas

O que e por quê: duas dataclasses definem o que o laço conhece. `PedidoFerramenta` carrega os argumentos **sempre como dicionário**, fazendo o parse da string JSON da OpenAI aqui dentro. `RespostaLLM` guarda também o turno cru do assistant, porque a etapa de devolução precisa reenviá-lo no formato nativo. Normalizar para ler, guardar o original para reenviar.

Acrescente ao final de `nucleo/providers.py`:

```
# =====  
# Aula 0.3 - Tool use normalizado para o loop.  
#
```

```

# A chat() da aula 0.1 continua intocada. Daqui para baixo entra o que o
# LOOP precisa: uma chamada que oferece ferramentas e devolve SEMPRE o
# mesmo objeto neutro, seja qual for o provider, e a funcao que monta as
# mensagens de devolucao no dialeto certo. A diferenca entre os providers
# morre neste arquivo; o laco em agente.py nunca ve dialeto.
# =====

import json
from dataclasses import dataclass
from typing import Any

@dataclass
class PedidoFerramenta:
    """Um pedido de ferramenta feito pelo modelo, ja normalizado."""
    id: str          # id do pedido; usado para parear o resultado depois
    nome: str        # nome da ferramenta pedida
    argumentos: dict # SEMPRE dict, mesmo na OpenAI (que manda string JSON)

@dataclass
class RespostaLLM:
    """Uma volta do modelo, normalizada."""
    texto: str          # texto que o modelo escreveu (pode ser "")
    pedidos: list       # list[PedidoFerramenta]; vazia = acabou
    mensagem_assistant: Any = None # o turno CRU do assistant, para o historico

def chat_ferramentas(provider, messages, ferramentas, system=None,
                      max_tokens=1024):
    """Igual a chat() da aula 0.1, porem oferecendo ferramentas ao modelo.

    Devolve sempre um RespostaLLM, seja qual for o provider.
    'ferramentas' e o dict {"anthropic": [...], "openai": [...]} com os
    schemas em cada dialeto (a aula 0.4 vai gerar isso automaticamente).
    """
    if provider == "anthropic":
        kwargs = {
            "model": config.ANTHROPIC_MODEL,
            "max_tokens": max_tokens,          # obrigatorio na Anthropic
            "tools": ferramentas["anthropic"],
            "messages": messages,
        }
    if system:
        kwargs["system"] = system              # parametro, nunca mensagem

    resp = _anthropic.messages.create(**kwargs)

    # A resposta vem como LISTA DE BLOCOS: separamos texto de pedidos.
    texto = ""
    pedidos = []
    for bloco in resp.content:
        if bloco.type == "text":
            texto += bloco.text
        elif bloco.type == "tool_use":
            # Na Anthropic o input JA e dicionario. Sem json.loads.
            pedidos.append(PedidoFerramenta(
                id=bloco.id, nome=bloco.name, argumentos=bloco.input))

    return RespostaLLM(
        texto=texto,
        pedidos=pedidos,
        # O turno do assistant volta CRU para o historico: a API exige
        # rever o proprio pedido antes de receber o resultado.
    )

```

```

        mensagem_assistant={"role": "assistant", "content": resp.content},
    )

if provider == "openai":
    msgs = list(messages) # nunca mutar a lista original
    if system:
        msgs = [{"role": "system", "content": system}] + msgs

    resp = _openai.chat.completions.create(
        model=config.OPENAI_MODEL,
        messages=msgs,
        tools=ferramentas["openai"],
        max_completion_tokens=max_tokens,
    )
    msg = resp.choices[0].message

    pedidos = []
    for tc in (msg.tool_calls or []):
        pedidos.append(PedidoFerramenta(
            id=tc.id,
            nome=tc.function.name,
            # Na OpenAI os argumentos chegam como STRING JSON.
            argumentos=json.loads(tc.function.arguments),
        ))

    return RespostaLLM(
        texto=msg.content or "",
        pedidos=pedidos,
        mensagem_assistant=msg, # a mensagem crua do SDK
    )

raise ValueError(f"Provider desconhecido: {provider}")

def mensagens_de_resultado(provider, resposta, resultados):
    """Monta as mensagens de devolucao no dialeto do provider.

    resposta -> o RespostaLLM da volta (traz o turno cru do assistant)
    resultados -> list[(PedidoFerramenta, str)], o texto ja serializado

    Devolve a lista de mensagens a ANEXAR ao historico. E a etapa 4 do
    ciclo da aula 0.2, agora escrita uma unica vez.
    """
    if provider == "anthropic":
        # Anthropic: reinsere o turno do assistant e devolve TODOS os
        # resultados num unico turno de user, um bloco por pedido.
        blocos = [
            {"type": "tool_result", "tool_use_id": pedido.id,
             "content": saida}
            for pedido, saida in resultados
        ]
        return [resposta.mensagem_assistant,
                {"role": "user", "content": blocos}]

    if provider == "openai":
        # OpenAI: reinsere o turno do assistant e devolve UMA mensagem
        # de role "tool" por pedido, pareada pelo tool_call_id.
        msgs = [resposta.mensagem_assistant]
        for pedido, saida in resultados:
            msgs.append({
                "role": "tool",
                "tool_call_id": pedido.id,
                "content": saida,
            })

```

```

return msgs

raise ValueError(f"Provider desconhecido: {provider}")

```

O que faz: `chat_ferramentas()` reaproveita os clientes e a configuração da aula 0.1 e devolve sempre um `RespostaLLM`. `mensagens_de_resultado()` é a etapa 4 do ciclo da aula 0.2, agora escrita uma única vez, com a regra de devolução de pedidos paralelos de cada provider.

Passo 2 — O laço

O que e por quê: toda a teoria cabe em poucas linhas. É por isso que esta aula existe. O laço devolve uma `Execucao`, não só uma string: resposta, número de voltas, motivo da parada e histórico. É o embrião do trace da aula 0.7.

Crie `nucleo/agente.py`:

```

# Aula 0.3 - O loop de agente, em Python puro.
#
# O laço não sabe qual provider está rodando nem quais ferramentas
# existem no sistema. Ele conhece apenas o ciclo:
#
#     CONTEXTO -> DECISAO -> ACAO -> OBSERVACAO -> (repete)
#
# A frase que resume a aula: o agente não é o modelo, o agente é o loop.
#
# Depende de (aulas anteriores):
#     nucleo/config.py      -> nomes de modelo                (aula 0.1)
#     nucleo/providers.py   -> chat_ferramentas(),
#                             mensagens_de_resultado() e as
#                             dataclasses PedidoFerramenta e
#                             RespostaLLM                    (aula 0.3)

import json                      # serializa o retorno da ferramenta
from dataclasses import dataclass # classe de dados sem boilerplate

from nucleo.providers import chat_ferramentas, mensagens_de_resultado

# -----
# O RESULTADO DE UMA EXECUCAO
#
# Não devolvemos só a string da resposta. Devolvemos também a prestação
# de contas do caminho percorrido. Isso é o embrião da observabilidade
# que vira trace no Langfuse na aula 0.7 e estado persistido na Fase 1.
# -----
@dataclass
class Execucao:
    """O que o agente devolve ao fim de uma tarefa."""

    resposta: str
    # Texto final que o agente produziu para o usuário.

    iteracoes: int
    # Quantas voltas o laço deu. Métrica direta de custo e de latência.

    parou_por: str
    # "resposta_final" ou "limite_de_iteracoes".
    # Saber POR QUE parou e o que permite diagnosticar agente travado.

    historico: list
    # A conversa completa: pergunta, decisões do modelo e observações.
    # Este é o ESTADO do agente. Hoje ele vive na memória e morre com o

```

```

# processo. Persistir isto e o tema de checkpointing na Fase 1.

# -----
# 0 LACO
# -----
def rodar_agente(
    pergunta,
    ferramentas,
    executores,
    provider="anthropic",
    system=None,
    max_iteracoes=6,
    verbose=True,
):
    """Roda o agente ate ele responder ou ate bater o teto de iteracoes.

    pergunta      -> texto do usuario que inicia a tarefa
    ferramentas   -> dict {"anthropic": [...], "openai": [...]} com os schemas
    executores     -> dict {nome_da_ferramenta: funcao_python}
    provider       -> "anthropic" ou "openai"
    system         -> instrucao de sistema, opcional
    max_iteracoes -> teto de voltas. E orcamento, nao paranoia.
    verbose        -> imprime o caminho percorrido, util em aula e depuracao
    """

    historico = [{"role": "user", "content": pergunta}]
    # O HISTORICO E O ESTADO. Nao existe memoria magica: o modelo
    # "lembra" porque reenviamos a conversa inteira a cada volta, com
    # um item a mais.

    # Usamos 'for' em vez de 'while True' de proposito: o teto de
    # iteracoes ja vem embutido na propria estrutura, entao e
    # impossivel esquecer dele.
    for iteracao in range(1, max_iteracoes + 1):

        # DECISAO: o modelo le o contexto e escolhe o proximo passo.
        resposta = chat_ferramentas(
            provider, historico, ferramentas, system=system)

        # PARADA 1: respondeu sem pedir ferramenta. Caso feliz.
        if not resposta.pedidos:
            if verbose:
                print(f"[volta {iteracao}] resposta final")
            return Execucao(
                resposta=resposta.texto,
                iteracoes=iteracao,
                parou_por="resposta_final",
                historico=historico,
            )

        # ACAO: executa cada pedido. Podem vir varios numa volta so.
        resultados = []
        for pedido in resposta.pedidos:
            if verbose:
                print(f"[volta {iteracao}] {pedido.nome}({pedido.argumentos})")

            funcao = executores.get(pedido.nome)
            if funcao is None:
                # Ferramenta desconhecida NAO derruba o agente: vira
                # observacao que ele le e contorna. E a semente da
                # resiliencia da aula 0.6.
                saida = {"erro": f"ferramenta desconhecida: {pedido.nome}"}
            else:

```

```

        saida = funcao(**pedido.argumentos)

        # 0 resultado e TEXTO que o modelo vai ler na proxima volta.
        # ensure_ascii=False preserva os acentos do portgues.
        resultados.append((pedido, json.dumps(saida, ensure_ascii=False)))

    # OBSERVACAO: o turno do assistant e os resultados voltam para o
    # historico, no dialeto certo. A proxima volta enxerga tudo.
    historico.extend(
        mensagens_de_resultado(provider, resposta, resultados))

# PARADA 2: estourou o teto. Isto NAO e sucesso: sinalizamos o
# motivo para quem chamou decidir o que fazer.
if verbose:
    print(f"[fim] limite de {max_iteracoes} iteracoes atingido")
return Execucão(
    resposta="",
    iteracoes=max_iteracoes,
    parou_por="limite_de_iteracoes",
    historico=historico,
)

```

O que faz: leia o corpo do laço de novo. Decisão, ação, observação: a teoria traduzida linha a linha, e o mesmo desenho que você vai reconhecer no `StateGraph` do LangGraph. Repare que ferramenta desconhecida não derruba nada: vira observação que o modelo lê e contorna. É a semente da resiliência da aula 0.6.

Passo 3 — Ver funcionando, em código descartável

O que e por quê: duas ferramentas mock e os schemas escritos nos dois dialetos, só para ver o laço encadeando. Este código vive em `scratch/`, que o `.gitignore` ignora: é rascunho, não produto. Os schemas duplicados continuam incomodando, e é de propósito: é a dor que a aula 0.4 elimina.

```

# scratch/rascunho_0_3.py (descartavel: o .gitignore ignora scratch/)
from nucleo.agente import rodar_agente

def buscar_info_lead(email):
    base = {"carlos@datalog.io": {"empresa": "DataLog", "plano": "pro", "mrr": 490}}
    return base.get(email, {"erro": "lead nao encontrado"})

def calcular_score_lead(plano, mrr):
    pontos = {"trial": 10, "pro": 40, "enterprise": 70}.get(plano, 0)
    return {"score": pontos + min(mrr // 20, 30)}

props_busca = {"email": {"type": "string", "description": "Email do lead"}}
props_score = {"plano": {"type": "string", "enum": ["trial", "pro", "enterprise"]},
               "mrr": {"type": "integer", "description": "Receita mensal"}}
desc_busca = "Busca empresa, plano e MRR de um lead pelo email."
desc_score = "Calcula o score 0-100 de um lead. Use DEPOIS de buscar os dados."

def ant(nome, desc, props, req):
    return {"name": nome, "description": desc,
            "input_schema": {"type": "object", "properties": props, "required": req}}

def oai(nome, desc, props, req):
    return {"type": "function", "function": {"name": nome, "description": desc,
                                             "parameters": {"type": "object", "properties": props, "required": req}}}

ferramentas = {
    "anthropic": [ant("buscar_info_lead", desc_busca, props_busca, ["email"]),

```

```

        ant("calcular_score_lead", desc_score, props_score, ["plano", "mrr"])),
    "openai": [oai("buscar_info_lead", desc_busca, props_busca, ["email"]),
              oai("calcular_score_lead", desc_score, props_score, ["plano", "mrr"])],
}
executores = {"buscar_info_lead": buscar_info_lead,
              "calcular_score_lead": calcular_score_lead}

for provider in ["anthropic", "openai"]:
    exe = rodar_agente("Qual o score do lead carlos@datalog.io?",
                      ferramentas, executores, provider=provider)
    print(provider, exe.parou_por, exe.iteracoes, "->", exe.resposta)

```

Resultado esperado: nas duas execuções, o laço imprime a busca na volta 1, o cálculo do score na volta 2 e a resposta final na volta 3, com `parou_por` igual a `resposta_final`. Você não disse a ordem: o modelo deduziu que precisava dos dados antes do score. Isso é o agente.

Passo 4 — Testar o laço sem chamar o modelo

O que e por quê: o truque é trocar `chat_ferramentas` por um roteiro pré-escrito de `RespostaLLM`. O laço roda de verdade; só as decisões do modelo são nossas. O helper:

```

def _roteiro(monkeypatch, respostas):
    """Troca chat_ferramentas por um roteiro pre-escrito de RespostaLLM.
    O laço roda de verdade; so as decisoes do modelo sao nossas."""
    fila = list(respostas)
    contador = {"n": 0}

    def fake(provider, messages, ferramentas, system=None, max_tokens=1024):
        contador["n"] += 1
        return fila.pop(0)

    monkeypatch.setattr(agente, "chat_ferramentas", fake)
    # mensagens_de_resultado tambem vira dublê simples: o formato real de
    # cada dialeto e testado separadamente, mais abaixo.
    monkeypatch.setattr(agente, "mensagens_de_resultado",
                        lambda provider, resp, res: [{"volta": len(res)}])

    return contador

```

E o primeiro dos sete testes, o do encadeamento:

```

# ----- 1. o laço encadeia duas ferramentas e para sozinho -----

def test_encadeia_duas_ferramentas_e_para(monkeypatch):
    chamadas = []
    contador = _roteiro(monkeypatch, [
        RespostaLLM("", [PedidoFerramenta("1", "buscar_info_lead",
                                           {"email": "carlos@datalog.io"})]),
        RespostaLLM("", [PedidoFerramenta("2", "calcular_score_lead",
                                           {"plano": "pro", "mrr": 490})]),
        RespostaLLM("Lead qualificado, score 70.", []),
    ])

    exe = agente.rodar_agente("0 lead e bom?", FERRAMENTAS,
                              _executores(chamadas), verbose=False)

    assert exe.parou_por == "resposta_final"
    assert exe.iteracoes == 3
    assert contador["n"] == 3
    assert "score 70" in exe.resposta
    # A ordem de execucao seguiu a decisao do modelo.

```

```
assert [c[0] for c in chamadas] == ["buscar_info_lead",  
                                     "calcular_score_lead"]
```

O arquivo completo cobre sete casos: encadeamento de duas ferramentas, limite de iterações sinalizado como falha, pedidos paralelos numa mesma volta, ferramenta desconhecida virando observação, normalização da Anthropic, normalização da OpenAI com parse da string JSON, e a devolução no formato de cada dialeto. **Resultado esperado:** `pytest` mostra `12 passed, 2 deselected`, sem rede e sem custo.

Esses testes provam que o laço encadeia, acumula, respeita o teto e trata bordas. Não provam, nem tentam provar, que o modelo escolheria bem as ferramentas. Essa é uma pergunta de avaliação, com instrumento próprio, na Fase 3.

Passo 5 — Commit e tag

```
pytest -v          # esperado: 12 passed, 2 deselected  
git status         # .env e scratch/ NAO podem aparecer  
git add .  
git commit -m "Aula 0.3: loop de agente em Python puro"  
git tag aula-0.3  
git push && git push --tags
```

5. Portabilidade

O `AgentExecutor` do LangChain, o `create_react_agent` do LangGraph e o runner do OpenAI Agents SDK são todos este laço, com mais tratamento de borda. Ao ler a documentação de qualquer um deles, procure as quatro condições de parada: estão lá, às vezes com outro nome (o `recursion_limit` do LangGraph é o seu `max_iteracoes`). E a separação entre laço e dialeto que você fez hoje é a mesma que permite a esses frameworks suportar dezenas de providers.

6. Terminei quando

O critério da aula: **o agente resolve uma tarefa usando duas ou mais ferramentas em sequência, por conta própria.**

- O rascunho encadeia busca e score nos dois providers, sem você indicar a ordem.
- Com `max_iteracoes=1`, o rascunho para com `limite_de_iteracoes` e resposta vazia, em vez de fingir sucesso.
- `pytest` mostra `12 passed, 2 deselected`.
- Você explica, sem olhar o código, por que o turno do assistant precisa voltar para o histórico.

7. Erros comuns e troubleshooting

- **A API rejeita a conversa na segunda volta.** O turno do assistant com o pedido não foi reenviado antes dos resultados. É o que o campo `mensagem_assistant` existe para garantir.
- **O agente ignora o segundo pedido de uma volta.** Laço assumindo um pedido por volta. Itere sobre `resposta.pedidos`, sempre.
- **Anthropic reclama de tool_result faltando com pedidos paralelos.** Todos os resultados vão num único turno de user, um bloco por pedido, cada um com o seu id.

- **O agente estoura o limite em ciclo.** Olhe o `historico` da `Execucao`. O padrão típico é a ferramenta devolver algo que o modelo não reconhece como resposta, e ele tenta de novo. O resultado da ferramenta é texto que o modelo lê: se for críptico, o agente se perde.
- **RuntimeError: Falta ANTHROPIC_API_KEY ao rodar o rascunho.** O rascunho precisa do `.env` real. Nos testes, quem injeta as chaves falsas é o `conftest.py`; fora do pytest, não.
- **Acentos viram sequências de escape no resultado.** Faltou `ensure_ascii=False` no `json.dumps`.

8. Ponte para a próxima aula

O laço funciona, mas você escreveu cada schema duas vezes, uma por dialeto, além da função Python. Com duas ferramentas incomoda; com as quinze de um sistema real, é dessincronia esperando a primeira refatoração distraída. Na **Aula 0.4, Registro de ferramentas**, um decorator lê a própria função e gera os dois schemas sozinho: adicionar ferramenta vira escrever uma função, e o laço de hoje não muda uma linha.